Eternity II

NOME: João Pedro Corrêa Silva **R.A:** 11202321629
NOME: João Pedro Sousa Bianchim **R.A:** 11201920729
PROFESSOR: Emilio Francesquini

Relatório do Projeto

Descrição

A implementação do código foi feita usando como base heurísticas que também foram usadas para a otimização da solução do problema sequencial desenvolvida pelo professor. Sendo elas: uma busca começando por um vértice (duas cores 0), indo primeiro pelas bordas do tabuleiro (preenchendo a matriz do tabuleiro numa forma de espiral) e filtrando as peças pelas cores antes de fazer a busca (cria-se um vetor para cada cor para otimizar a busca por peças que encaixem).
Com esse funcionamento, a divisão de tarefas foi organizada segundo o modelo master-worker: o processador de rank 0 atua como master, responsável por gerenciar a execução geral e escutar se algum outro processo encontrou uma solução válida. Caso isso ocorra, ele comunica aos demais workers (ranks de 1 a no máximo 9) para interromperem suas execuções.
Cada worker recebe um vértice inicial e uma direção de busca (horária ou anti-horária), sendo essa configuração determinada com base no seu rank. A partir disso, cada processo inicia a busca por uma solução válida. Nesse processo de busca, ele de vez em quando para para checar se houve uma mensagem do master informando se deve ou não continuar a busca (fazendo com que todos parem um tempo aproximadamente igual), se encontrar uma solução válida, a envia a P0 e encerra sua execução.
Dada a forma como o algoritmo está estruturado, há uma igualdade no balanceamento de carga dos trabalhadores, todos processam o jogo inteiro até que algum encontre a solução.

Resultados

A seguir, apresentamos os resultados obtidos com a execução dos algoritmos. Primeiramente, utilizamos entradas fornecidas pelo professor, as quais variam em número de cores e dimensões:

Paralelo rodando com 6 Processadores, entradas fornecidas do professor							
	EX: 00(2x2 - 3 cores)	**EX: 01(4x4 - 5 cores)**	**EX: 02(4x4 - 5 cores)**	**EX: 03(4x4 - 5 cores)**	**EX: 04(4x4 - 5 c**	**EX: 05(4x4 - 5 c**	**EX: 06(4x4 - 5 cores)**
speedup	9.5	3.84	6.37	3.04	29.95	24.59	11.77
eficiencia	1.583333333	0.64	1.061666667	0.5066666667	4.991666667	4.098333333	1.961666667

Observa-se que o speedup obtido foi, em geral, positivo, chegando a valores bastante altos, como 29.95 e a eficiência manteve-se boa ao longo dos testes.

Contudo para uma análise mais controlada, realizamos novos experimentos com entradas geradas automaticamente, mantendo constante o número de processadores (6 núcleos). Nessas simulações, variamos o tamanho da entrada, mantendo constante o número de cores:

Paralelo rodando com 6 Processadores, entradas geradas pelo gerador, 6 cores variam tamanho							
	2X2	**3X3**	**4X4**	**5X5**	**6X6**	**7X7**	**8X8**
speedup	8.25	7.3	4.85	1.61	0.72	1.27	8.24
eficiencia	1.375	1.216666667	0.8083333333	0.2683333333	0.12	0.2116666667	1.373333333

Nesses novos testes, o *speedup* permaneceu positivo, mas a eficiência caiu muito conforme o problema aumentava. Ainda assim, vemos que a eficiência voltou a estar positiva no caso do 8x8.

Decidimos também fazer uma análise mais aprofundada da eficiência, verificando como ela se alterava ao aumentar o tamanho do problema para diferentes quantidades de processadores, a seguir o resultado:

	Tamanho do problema					
N_processadores	2x2	3x3	4x4	5x5	6x6	7x7
1	1	1	1	1	1.00	1
2	7.70	6.09	3.58	2.35	0.39	0.49
3	5.13	4.06	2.39	1.16	0.27	0.34
4	3.21	5.58	2.26	0.64	0.17	0.23
5	3.08	4.47	1.56	0.57	0.09	0.18

Vê-se neste último gráfico que não há escalabilidade, mas ainda há um speedup (não garantido). Isso já era esperado visto que a depender do problema o sequencial pode “ter sorte” e executar em uma velocidade parecida ou igual a do paralelo.